

# Retrieval-Augmented Generation

Ground an LLM response in selected evidence retrieved from an external document collection.



## WHY THIS MATTERS

# Start from what you already know

LLMs, vectors, dot products, application pipelines, schemas, and evaluation

- 01 Explain document processing and chunking in plain language.
- 02 Use embeddings in a small worked example.
- 03 Complete the guided lab and verify grounded generation and citations.

## CORE CONCEPT

# The vocabulary of this topic

---

Ground an LLM response in selected evidence retrieved from an external document collection.

**document processing and chunking:** Document processing extracts structured content; chunking divides it into retrievable evidence units.

**embeddings:** Embeddings map content into fixed-length vectors whose geometry can approximate semantic similarity.

**vector search:** Vector search ranks stored embeddings by geometric similarity to an embedded query.

**grounded generation and citations:** Grounded generation uses supplied evidence; citations identify which evidence supports each claim.

## FOUR CONNECTED IDEAS

# How the concept works

01

**document  
processing and  
chunking**



02

**embeddings**



03

**vector search**



04

**grounded  
generation and  
citations**

$\cos(a,b)=(a \cdot b)/(\|a\| \|b\|)$ ; retrieve similar chunks, assemble context, then generate only from that context.

## CORE CONCEPT

# Read every part of the mechanism

---

$\cos(a,b)=(a \cdot b)/(\|a\| \|b\|)$ ; retrieve similar chunks, assemble context, then generate only from that context.

**chunk:** A retrievable passage plus metadata.

**embedding:** A semantic vector.

**citation:** A reference connecting a claim to evidence.

## WORKED EXAMPLE

## Worked example by hand

For vectors  $[1,0]$  and  $[0.8,0.6]$ , cosine similarity is 0.8; rank it above  $[0,1]$ , whose score is 0.

01

---

Known values

02

---

Apply the mechanism

03

---

Interpret the result

RUN IT AND INSPECT THE RESULT

## Map the concept to Python

```
import numpy as np
a, b = np.array([1, 0]), np.array([0.8, 0.6])
score = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(score)
```

OUTPUT

0.8

Each line corresponds to a concept already explained.

## QUALITY GATE

# Build the complete mental model



**Chunk around meaningful document structure and retain source, page, section, and access metadata.**

---



**Embed both chunks and queries into the same vector space before similarity comparison.**

---



**Retrieval selects evidence; generation must remain constrained to that supplied evidence.**

---



**Refuse or state insufficient evidence when no retrieved passage supports an answer.**

---



## FOUR CONNECTED IDEAS

# Notebook readiness map

01

**document  
processing and  
chunking**

02

**embeddings**

03

**vector search**

04

**grounded  
generation and  
citations**



Use the concepts from this lesson to read and complete  
<https://github.com/curiously/Al-Bootcamp/blob/master/17.rag-from-scratch.ipynb>.

## RISK REVIEW

# Common beginner mistakes

**01** Choosing chunk sizes without inspecting document structure.

---

**02** Accepting a fluent answer that has no supporting retrieved evidence.

---

## GUIDED LAB

**Parse a document, create chunks and embeddings, retrieve evidence, and produce a cited answer or refusal.**

01

02

03

04

---

**Predict**

---

**Run**

---

**Inspect**

---

**Explain**

## SESSION SYNTHESIS

# Keep the system, not isolated definitions.

01 Explain document processing and chunking in plain language.

02 Use embeddings in a small worked example.

03 Complete the guided lab and verify grounded generation and citations.

## NEXT EVIDENCE

**A from-scratch RAG notebook with inspectable chunks, similarity scores, citations, and unanswerable tests.**